TCP/IP PROTOCOL BASED CONVERTER CONTROL LOGIC DESIGN AND TESTING

A PROJECT REPORT

submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRICAL AND ELECTRONICS ENGINEERING

by

ABHINAV SANTOSH KUMAR (23BEE1023)
TANMAY KUMAR (23BEE1105)
ANIRUDH ALLALA (23BEE1111)

under the guidance of

Dr. Pritam Bhowmik

SCHOOL OF ELECTRICAL ENGINEERING
VIT CHENNAI

NOVEMBER 2026

CERTIFICATE

This is to certify that the project work titled **"TCP/IP PROTOCOL BASED CONVERTER CONTROL LOGIC DESIGN AND TESTING"** submitted by **Abhinav Santosh Kumar (23BEE1023), Tanmay Kumar (23BEE1105) and Anirudh Allala (23BEE1111)** is in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY**, is a record of bona fide work done under my guidance. The contents of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. Pritam Bhowmik
Project Supervisor

The thesis is satisfactory / unsatisfactory

Internal Examiner **External Examiner**

Approved by

Head of the Department
B.Tech Electrical and Electronics Engineering

ACKNOWLEDGEMENT

We would like to thank our guide, Dr. Pritam, School of Electrical Engineering, VIT Chennai, for his guidance and patience through the project. His comments at each review helped us tighten the design and plan the work properly.

We are also grateful to the Head of the Department of B.Tech Electrical and Electronics Engineering, the faculty of the School of Electrical Engineering and the laboratory staff who helped us with the facilities for building and testing the prototype.

We thank the Management of VIT Chennai and the Director of the School of Electrical Engineering for the opportunity to carry out this work at the University, and our families and friends for their support.

Abhinav Santosh Kumar (23BEE1023)
Tanmay Kumar (23BEE1105)
Anirudh Allala (23BEE1111)

ABSTRACT

When batteries in parallel are at different states of charge (SoC), they age unevenly, give less usable capacity and waste energy. Fixed hardware balancing circuits cope poorly with a large initial difference in charge, especially while the batteries are also supplying a load. In this project each battery is connected to the load through a DC–DC buck converter, and an intelligent controller sets the duty cycle of each converter so that the SoC of the batteries evens out. In the final system, voltage and current readings go from an Arduino Q to a host computer over TCP/IP. A machine-learning model on the computer returns a duty cycle, and the Arduino Q generates the matching PWM gate pulse.

What has been finished up to Review 2 is the converter itself and its closed-loop control. We designed a 24 V to 12 V, 2 A synchronous buck converter for 50 kHz switching with a 0.6 A inductor ripple target, ±0.05 V output ripple and at least 80 % efficiency. It uses IRLZ44N switches, a 330 µH inductor, 470 µF capacitors and an IR2104 gate driver. We analysed the design, simulated it in MATLAB/Simulink and at circuit level including the switching transients, laid it out on a PCB and built it. An ESP32 running a 10 kHz PI controller with anti-windup regulates the output at 12 V. Measurements at 50 kHz and 30 kHz show the gate and drain switching, how the output follows the duty cycle, the ringing and the gate-driver delay. The TCP/IP link, the ML duty-cycle computation and the SoC equalisation tests are the remaining work.

TABLE OF CONTENTS

LIST OF TABLES vii
LIST OF FIGURES viii
ABBREVIATIONS AND NOMENCLATURE ix
CHAPTER I 1
1. INTRODUCTION 1
1.1 INTRODUCTION 1
1.1.1 Motivation 1
1.1.2 Objectives 1
1.1.3 Scope of the Work 2
1.2 ORGANIZATION OF THESIS 2
CHAPTER II 3
2. PROJECT DESCRIPTION 3
2.1 OVERVIEW OF PROJECT 3
2.2 MODULES OF THE PROJECT 3
2.2.1 Module 1: Power Stage (Synchronous Buck Converter) 3
2.2.2 Module 2: Gate Drive and Closed-Loop Control 4
2.2.3 Module 3: TCP/IP Communication and ML-Based Duty Cycle (Planned) 4
2.3 TASKS AND MILESTONES 5
CHAPTER III 6
3. DESIGN OF TCP/IP BASED CONVERTER CONTROL 6
3.1 DESIGN APPROACH 6
3.1.1 Codes and Standards 6
3.1.2 Realistic Constraints 6
3.1.3 Alternatives and Tradeoffs 7
3.2 DESIGN SPECIFICATIONS 7
CHAPTER IV 11
4. PROJECT DEMONSTRATION 11
4.1 INTRODUCTION 11
4.2 ANALYTICAL RESULTS 11
4.3 SIMULATION RESULTS 11
4.4 HARDWARE RESULTS 13
4.4.1 PCB Design and Prototype 13
4.4.2 Switching Waveforms 15
4.4.3 Input and Output Voltage with Varying Duty Cycle 15
4.4.4 Ringing 16
4.4.5 Gate Driver Delay and Dead Time 16
4.4.6 Closed-Loop Operation 16
CHAPTER V 17
CONCLUSION 17
5.1 COST ANALYSIS 17
5.2 SCOPE OF WORK 17

5.3 SUMMARY 18
REFERENCES 19
PUBLICATIONS BASED ON THESIS 20
APPENDICES 21
Appendix 1: ESP32 Closed-Loop PI Control Firmware 21
Appendix 2: Simulation Model Description 23
CURRICULUM VITAE 24

LIST OF TABLES

Table 2.1: Tasks and milestones 5
Table 3.1: Design specifications of the buck converter 7
Table 3.2: Components used in the power and driver circuits 7
Table 3.3: Controller and firmware parameters 9
Table 4.1: Summary of analytical results 11
Table 5.1: Bill of materials and cost 17

LIST OF FIGURES

Figure 2.1: Planned architecture of the SoC-balancing system 3
Figure 2.2: Closed-loop buck converter with ESP32 PI control 4
Figure 4.1: MATLAB/Simulink closed-loop buck converter model 12
Figure 4.2: Simulated start-up transient 12
Figure 4.3: Simulated steady-state output voltage and inductor current 12
Figure 4.4: Simulated turn-on switching transient 13
Figure 4.5: Simulated turn-off switching transient 13
Figure 4.6: PCB schematic: driver circuit and power circuit 14
Figure 4.7: PCB layout 14
Figure 4.8: PCB 3D view 14
Figure 4.9: Assembled hardware prototype 15
Figure 4.10: Low-side Vgs and Vds at 50 kHz 15
Figure 4.11: Low-side Vgs and Vds at 30 kHz 15
Figure 4.12: Input and output voltage for changing duty cycle 15
Figure 4.13: Ringing close-up of input and output voltage 16
Figure 4.14: ESP32 PWM pulse and gate driver output pulse 16

ABBREVIATIONS AND NOMENCLATURE

ADC	Analog-to-Digital Converter
CCM	Continuous Conduction Mode
DC	Direct Current
ERC	Electrical Rules Check
DRC	Design Rules Check
GPIO	General-Purpose Input/Output
IP	Internet Protocol
LC	Inductor–Capacitor
ML	Machine Learning
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
PCB	Printed Circuit Board
PI	Proportional–Integral
PWM	Pulse-Width Modulation
SoC	State of Charge
TCP	Transmission Control Protocol
D	Duty cycle
f_s	Switching frequency
L, C	Inductance, capacitance
ΔI_L	Inductor current ripple
V_{in}, V_o	Input and output voltage

CHAPTER I

1. INTRODUCTION

1.1 INTRODUCTION

When a single battery cannot give enough capacity or current, several are connected in parallel. The problem is that no two batteries are really the same. They differ in age, internal resistance, temperature and, most often, in how full they are when they are connected. If two batteries at different states of charge (SoC) feed one load directly, the fuller one does most of the work. The pack then ages unevenly, the usable capacity falls and part of the stored energy is never used.

Hardware balancing circuits, whether they bleed charge through resistors or move it between cells, do one fixed thing. They do not cope well with a large gap in initial charge, particularly when the pack is also supplying a load. We wanted to see whether the job could be done in software instead. In our approach every battery feeds the shared load through its own DC–DC converter, and the duty cycle of each converter decides how much current its battery gives. If the duty cycles are chosen well, the batteries run down towards the same SoC.

The full system sends voltage and current readings from an Arduino Q to a host computer over TCP/IP. A machine-learning (ML) model on the computer turns the readings into a duty cycle and sends it back, and the Arduino Q produces the PWM gate pulse from it. At the time of Review 2 we have finished the first piece of this plan, a closed-loop DC buck converter. This report describes that converter and shows where it fits in the larger system.

1.1.1 Motivation

Uneven SoC in parallel batteries speeds up degradation, cuts the usable capacity and wastes energy. A fixed hardware circuit cannot adapt to the condition of the pack. With converters, the share of load each battery takes becomes a software decision, so the same hardware can handle different starting charges and different loads. Putting the decision on a host computer also lets us try data-driven methods that would be too heavy for a small microcontroller.

1.1.2 Objectives

The project has the following objectives.

- Design a 24 V to 12 V, 2 A DC buck converter switching at 50 kHz, with an inductor current ripple of 0.6 A, output ripple of ±0.05 V and efficiency of at least 80 %.
- Model the converter analytically, in MATLAB/Simulink and at circuit level, and look at its switching transients.
- Design a PCB and build a prototype with a half-bridge gate driver.
- Regulate the output voltage in closed loop with a PI controller running on an ESP32.
- Set up a TCP/IP link so that voltage and current can be sent from the Arduino Q to a host computer and a duty cycle sent back.
- Use an ML model on the host computer to compute the duty cycle so that batteries behind different converters end up at the same SoC.

1.1.3 Scope of the Work

This semester's work, up to Review 2, covers the first four objectives: designing, simulating, building and closing the loop on the DC buck converter. The TCP/IP link, the ML model and the multi-converter SoC equalisation are the final goal and are still to be done. We describe them here because the converter was designed with that architecture in mind.

1.2 ORGANIZATION OF THESIS

Chapter I states the problem and the objectives. Chapter II gives an overview of the project, its modules and its milestones. Chapter III covers the design approach, the specifications and the calculations using the components actually fitted on the board. Chapter IV presents the analytical, simulation and hardware results. Chapter V gives the cost, the remaining work and a summary. The references, two appendices (the ESP32 firmware and a description of the Simulink model) and the team's CVs come at the end.

CHAPTER II

2. PROJECT DESCRIPTION

2.1 OVERVIEW OF PROJECT

The aim is to keep batteries behind different converters at the same SoC. If one battery holds more charge than another, its converter is told to take more energy from it, so that over time both batteries reach the same level while the load keeps running. The duty cycle of each converter is the quantity that sets this share.

Figure 2.1 shows the final architecture. Each battery feeds the shared load through a gate driver and a buck converter. The Arduino Q measures the voltage and current of each channel and generates the PWM gate signals. The measurements go to a host computer over a wireless TCP/IP link, where the ML model works out the duty cycles and returns them.

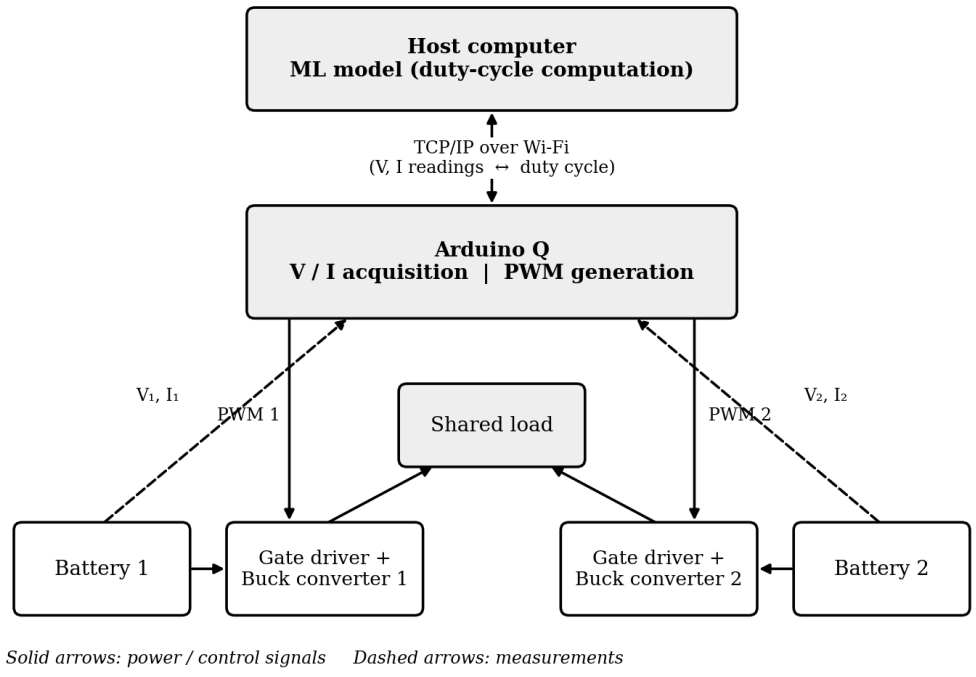


Figure 2.1: Planned architecture of the SoC-balancing system

2.2 MODULES OF THE PROJECT

We split the project into three modules. Modules 1 and 2 together make up the closed-loop buck converter that is already working. Module 3 is the communication and ML layer that will be added next.

2.2.1 Module 1: Power Stage (Synchronous Buck Converter)

The power stage steps 24 V down to 12 V at up to 2 A. Two IRLZ44N MOSFETs are used, Q2 on the high side and Q1 on the low side. The low-side device acts as the rectifier and conducts while Q2 is off, so there is no diode drop to lose power in. A 330 µH inductor (L1) and a 470 µF electrolytic capacitor (C4) filter the switching-node voltage into the DC output. A second 470 µF capacitor (C3) sits across the input. Both the input and the output are brought out on screw terminals.

2.2.2 Module 2: Gate Drive and Closed-Loop Control

An IR2104 half-bridge driver turns one PWM input into the two gate signals. It is powered from a 12 V supply. The high-side gate is driven from a bootstrap circuit made of a 1N4148 diode (D1) and a 0.1 µF capacitor (C2), and each gate has a 33 Ω series resistor. A four-pin connector (J2) brings in the PWM signal, the shutdown signal, +12 V and ground. The IR2104 inserts a short dead time between the two gate signals so that both MOSFETs are never on together. The bootstrap capacitor is only recharged while the low-side MOSFET is on, so the firmware limits the duty cycle to 90 %.

An ESP32 closes the voltage loop. A 10 kΩ and 2.2 kΩ divider scales the output voltage down for the 12-bit ADC on GPIO34. A discrete PI controller running at 10 kHz computes the duty cycle, which leaves GPIO18 as a 50 kHz, 10-bit PWM and passes through a level shifter to the IR2104 input. GPIO19 drives the shutdown pin through the same level shifter. Figure 2.2 shows the loop.

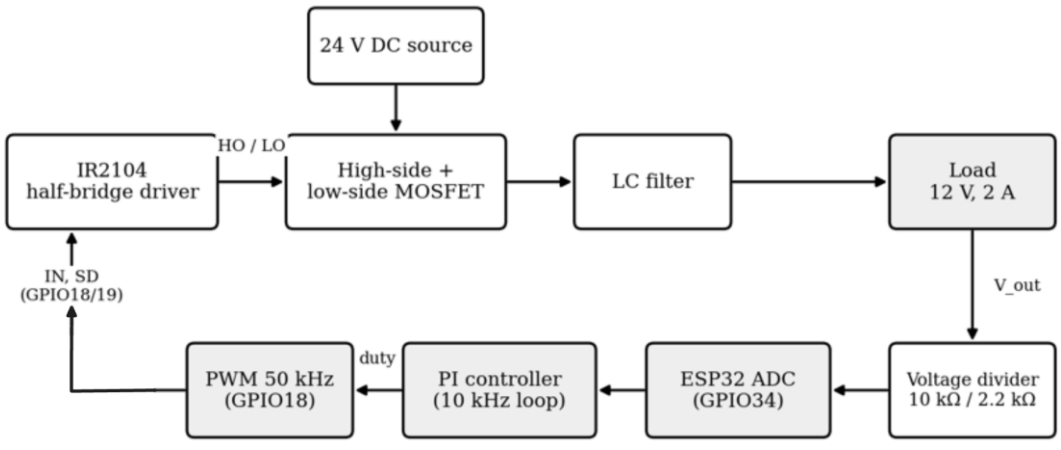


Figure 2.2: Closed-loop buck converter with ESP32 PI control

2.2.3 Module 3: TCP/IP Communication and ML-Based Duty Cycle (Planned)

In this module the measurement and PWM generation will move to the Arduino Q, which will send voltage and current readings to the host computer over TCP/IP. TCP was chosen because it delivers data in order and without loss. The ML model on the computer will take the readings from the converters and return a duty cycle for each one, and the Arduino Q will turn it into the gate pulse. The converter built in Modules 1 and 2 is the plant this layer will control.

2.3 TASKS AND MILESTONES

Table 2.1 lists the main tasks and where each one stands at Review 2.

Table 2.1: Tasks and milestones

S. No.	Task	Description	Status
1	Literature survey	SoC imbalance in parallel batteries, balancing methods, buck converter design	Completed
2	Specification and analysis	Converter specifications; calculation of ripple, losses and filter values	Completed
3	MATLAB/Simulink model	Closed-loop buck converter with PI controller	Completed
4	Circuit simulation	Output waveforms and switching transients	Completed
5	PCB design	Schematic, electrical rules check, layout, design rules check, 3D view	Completed
6	Prototype and testing	Switching at 50 kHz and 30 kHz, duty-cycle sweep, ringing, driver delay	Completed
7	Closed-loop control	ESP32 PI controller with anti-windup, 12 V regulation	Completed
8	TCP/IP link	Wireless data transfer between the host computer and the Arduino Q	Planned
9	ML duty-cycle model	Model on the host computer that returns a duty cycle from V and I	Planned
10	SoC equalisation	Tests with batteries at different SoC on several converters	Planned

CHAPTER III

3. DESIGN OF TCP/IP BASED CONVERTER CONTROL

3.1 DESIGN APPROACH

We started from the specifications, worked out the filter and switch requirements, and then chose the gate driver and the control scheme. The design was checked by calculation, by simulation and finally on the hardware. The controller is kept on a local microcontroller at this stage so that we can study the converter on its own before adding the ML layer.

3.1.1 Codes and Standards

The standards that apply to this work are listed below.

- IPC-2221 for printed board design. It guided trace widths and clearances and is the basis of the design rules check.
- IEC 62133-2 for the safety of lithium cells and batteries, which matters for the batteries the converters will eventually be connected to.
- IEC 62368-1 for the safety of electronic equipment.
- RFC 791 (IP) and RFC 793 (TCP) for the communication layer, along with IEEE 802.11 for the wireless link.

3.1.2 Realistic Constraints

- **Efficiency and heat.** An efficiency of 80 % at 24 W leaves only 6 W for all the losses.
- **Bootstrap supply.** The high-side driver can only be recharged when the low-side MOSFET conducts, so the duty cycle has to stay below 100 %. We set the limit at 90 %.
- **Switching noise.** Fast edges make the switching node ring, which we saw on the scope. Layout and the gate resistors both matter here.
- **Measurement accuracy.** The ESP32 ADC is limited in resolution and is noisy, which limits how closely the output can be held at 12 V.
- **Communication delay.** TCP over Wi-Fi has a variable delay. Fast switching and protection must therefore stay on the local controller, and the host computer can only send slower supervisory commands.
- **Cost and safety.** We used common, cheap parts. The batteries will need their own overcurrent and under-voltage protection.

3.1.3 Alternatives and Tradeoffs

- **Synchronous or diode buck.** A diode is simpler but at 2 A it wastes more power. We chose the synchronous design, and one half-bridge driver handles both switches.
- **Converters or passive balancing.** Resistive bleeding is simple but turns the extra charge into heat and cannot move it anywhere. Converters are more work but can be controlled in software.
- **Switching frequency.** We ran the converter at 50 kHz and also tested 30 kHz. A higher frequency allows smaller filter parts but increases switching loss and ringing.
- **TCP or UDP.** UDP is faster but can lose or reorder packets. We picked TCP for its reliability and accept the extra delay, since the supervisory loop is slow.
- **PI or ML.** A PI controller is enough for regulating the voltage. Deciding the duty cycles for SoC equalisation depends on the state of several batteries, which is where an ML model can help.

3.2 DESIGN SPECIFICATIONS

Table 3.1 gives the specifications of the converter.

Table 3.1: Design specifications of the buck converter

S. No.	Parameter	Specification	Unit
1	Input voltage	24	V
2	Input voltage ripple	1 % (24 V ± 0.24 V)	–
3	Output voltage	12	V
4	Output voltage ripple	± 0.05	V
5	Maximum output current	2	A
6	Inductor current ripple	0.6	A
7	Efficiency	≥ 80	%
8	Switching frequency	50 (30 also tested)	kHz

Table 3.2 lists the components fitted on the board, taken from the schematic in Figure 4.6.

Table 3.2: Components used in the power and driver circuits

Reference	Part / value	Role
Q2	IRLZ44N MOSFET	High-side switch
Q1	IRLZ44N MOSFET	Low-side switch (synchronous rectifier)

Reference	Part / value	Role
L1	330 µH	Output filter inductor
C3	470 µF electrolytic	Input capacitor
C4	470 µF electrolytic	Output capacitor
U1	IR2104	Half-bridge gate driver
D1	1N4148	Bootstrap diode
C1	0.1 µF	Supply decoupling for U1
C2	0.1 µF	Bootstrap capacitor
R1, R2	33 Ω each	Gate resistors for LO and HO
J1, J3	Screw terminals	Input and output connections
J2	4-pin connector	SD_IN, +12 V, PWM_IN, GND

Duty cycle and load. For an ideal buck converter in continuous conduction the duty cycle is

$$D = V_o / V_{in} = 12 / 24 = 0.5 \tag{3.1}$$

At 12 V and 2 A the smallest load resistance is 6 Ω and the output power is 24 W.

Inductor. With L = 330 µH as chosen, the inductor current ripple is

$$\Delta I_L = V_o(1 - D) / (L f_s) = 12 \times 0.5 / (330 \times 10^{-6} \times f_s) \tag{3.2}$$

This gives 0.364 A at 50 kHz and 0.606 A at 30 kHz. The 0.6 A ripple in the specification is therefore met at both frequencies, and at 30 kHz the ripple is almost exactly the specified value. For comparison, a ripple of 0.6 A at 50 kHz would only need 200 µH, so the inductor on the board has some margin. The peak inductor current is $I_{o,max} + \Delta I_L/2$, which is 2.18 A at 50 kHz and 2.30 A at 30 kHz. The inductor must be rated above these values.

Output capacitor. Ignoring ESR, the ripple voltage across chosen C4 = 470 µF is

$$\Delta V_o = \Delta I_L / (8 f_s C) \approx 1.9\ mV\ (50\ kHz),\ \ 5.4\ mV\ (30\ kHz) \tag{3.3}$$

This is far below the allowed 0.1 V peak to peak. For an electrolytic capacitor the ESR is what really decides the ripple, and to stay inside the limit the ESR must be below ΔV_o / ΔI_L, which is about 275 mΩ at 50 kHz and 165 mΩ at 30 kHz. The datasheet ESR of the capacitor used should be checked against these numbers.

Input capacitor. The pulsed input current produces a ripple on the 24 V rail of about I_o $D(1 - D) / (f_s C3) = 2 \times 0.25 / (50\,000 \times 470 \times 10^{-6}) \approx 21$ mV, against an allowed 0.48 V peak to peak (±0.24 V).

Continuous conduction. The inductor current stays continuous while the load current is above $\Delta I_L/2$. That is 0.18 A at 50 kHz, so the converter stays in continuous mode for loads up to about 66 Ω. At 30 kHz the limit is about 40 Ω.

LC filter. The corner frequency of L1 and C4 is

$$f_0 = 1 / (2\pi\sqrt{(LC)}) = 1 / (2\pi\sqrt{(330\ \mu H \times 470\ \mu F)}) \approx 404\ Hz \qquad (3.4)$$

which is about 120 times below the 50 kHz switching frequency, so the switching ripple is attenuated strongly. The characteristic impedance is $\sqrt{(L/C)} = 0.84\ \Omega$. With a 6 Ω load the quality factor is roughly $R/\sqrt{(L/C)} \approx 7$, so the filter is lightly damped. This explains the oscillation seen when the output capacitor charges at start-up and also limits how aggressive the control gains can be.

Switches. The IRLZ44N is a logic-level MOSFET rated for 55 V, comfortably above 24 V plus ringing. Its on-resistance is roughly 22 mΩ, so with an RMS current near $I_o\sqrt{D} \approx 1.4$ A each switch dissipates only about 0.04 W in conduction. Most of the loss budget therefore goes into switching and gate-drive loss. The 33 Ω gate resistors slow the edges to reduce ringing, which costs some extra switching loss.

Loss budget. At 24 W output and 80 % efficiency the input power can be at most 30 W, the average input current at most 1.25 A and the total loss at most 6 W.

Feedback scaling. The divider of $R_1 = 10$ kΩ and $R_2 = 2.2$ kΩ has a ratio of $(R_1 + R_2) / R_2 = 5.545$. So 12 V at the output shows up as 2.16 V at the ADC pin, and the 3.3 V ADC range covers outputs up to 18.3 V. One ADC step of 3.3 / 4095 = 0.806 mV corresponds to 4.47 mV at the output.

PWM resolution. A 10-bit PWM at 50 kHz has 1024 duty steps, each 0.098 % of the period. With a 24 V input that is about 23 mV of output voltage per step.

Controller. The controller is a discrete PI loop executed every $T_s = 100$ µs (10 kHz):

$$d[k] = K_p\, e[k] + K_i \Sigma\, e[j], \quad e[k] = V_{ref} - V_{out}[k] \qquad (3.5)$$

Here $V_{ref} = 12$ V, $K_p = 0.05$ duty per volt and $K_i = 0.01$ per sample. In continuous terms the integral gain is $K_i/T_s = 100\ s^{-1}$, which puts the PI zero at 2000 rad/s, or about 318 Hz. The plant gain is $V_{in} = 24$ V per unit of duty. Since the LC resonance is only about 404 Hz and lightly damped, the gains have to stay modest, and this is why the firmware starts with low values that are tuned on the hardware. The duty cycle is limited to between 0 and 90 %. When the output saturates, the integration for that sample is undone (at the lower limit

only if the error is negative) so that the integrator does not wind up. Table 3.3 lists the firmware settings.

Table 3.3: Controller and firmware parameters

Parameter	Value	Pin / note
PWM frequency	50 kHz	GPIO18
PWM resolution	10 bit (0 – 1023)	–
Driver shutdown (SD)	Enabled after start-up	GPIO19
ADC resolution / reference	12 bit / 3.3 V	GPIO34
Voltage divider	10 kΩ / 2.2 kΩ	Ratio 5.545
Control loop rate	10 kHz (100 µs)	Timed with micros()
Set-point	12 V	–
K_p, K_i	0.05, 0.01	Tuned on hardware
Duty limits	0 % – 90 %	Bootstrap recharge
Telemetry rate	10 Hz	Serial plotter, 115200 baud

CHAPTER IV

4. PROJECT DEMONSTRATION

4.1 INTRODUCTION

This chapter presents what we obtained by calculation, simulation and measurement on the prototype up to Review 2. The calculations check the design, the simulations look at the steady-state and switching behaviour, and the hardware results show the built converter under ESP32 control.

4.2 ANALYTICAL RESULTS

Table 4.1 collects the main calculated values for the fitted components (L = 330 μH, C = 470 μF) at the two switching frequencies used in testing.

Table 4.1: Summary of analytical results

Quantity	50 kHz	30 kHz
Inductor current ripple	0.364 A	0.606 A
Peak inductor current	2.18 A	2.30 A
Output ripple from C4 (ideal)	1.9 mV	5.4 mV
ESR limit for 0.1 V ripple	275 mΩ	165 mΩ
Input ripple from C3 (ideal)	21 mV	35 mV
Continuous conduction down to	0.18 A (66 Ω)	0.30 A (40 Ω)
LC corner frequency	404 Hz	404 Hz

The ideal duty cycle for 12 V is 0.5. In a real converter the switch drops, the dead time and the switching losses all pull the output down, so a slightly higher duty cycle is needed. We saw this in the open-loop simulation described next, and it is the main reason a feedback loop is needed.

4.3 SIMULATION RESULTS

We first modelled the converter in MATLAB/Simulink. Figure 4.1 shows the closed-loop model. A 24 V source feeds a high-side MOSFET and a low-side MOSFET with its body diode, and the low-side gate gets the inverse of the high-side signal through a NOT gate. The inductor, output capacitor and load resistor form the output stage, with blocks that measure the inductor current and the output voltage. The output voltage is subtracted from

a 12 V reference, the error goes to a PI controller with saturation, and a PWM generator turns the result into the gate signal.

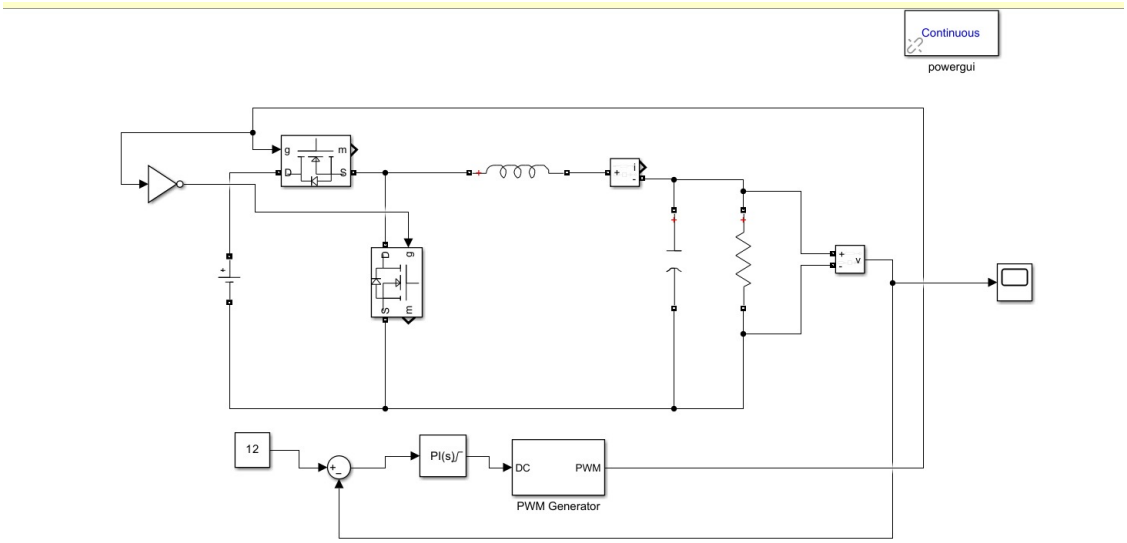


Figure 4.1: MATLAB/Simulink closed-loop buck converter model

A circuit-level simulation of the power stage was then used to look at the waveforms. At start-up a surge of current charges the empty output capacitor and sets the under-damped LC filter ringing briefly before the output settles (Figure 4.2). In steady state the filter turns the square wave at the switching node into a steady DC voltage. The inductor carries a continuous triangular ripple current, and the capacitor smooths it into a nearly flat output of about 11 V in the open-loop case (Figure 4.3). The gap from 12 V is one more reason to use feedback.

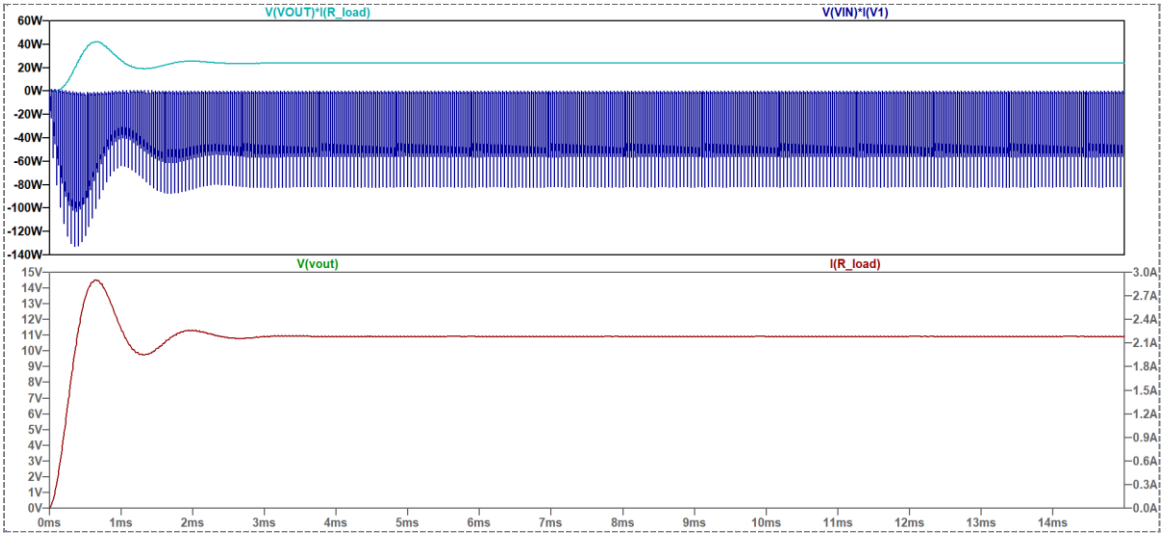


Figure 4.2: Simulated start-up transient

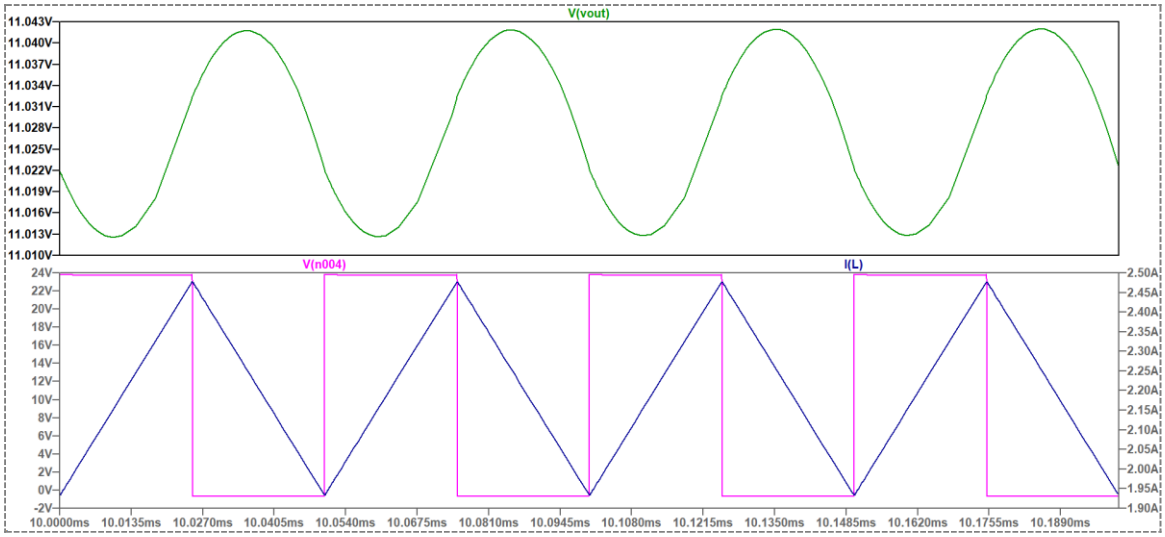


Figure 4.3: Simulated steady-state output voltage and inductor current

We also simulated the MOSFET switching transients. At turn-on there is a short spike of power loss because the drain-source voltage is still high while a large current flows in to charge the parasitic capacitances (Figure 4.4). At turn-off the loss spikes again while the gate capacitance discharges through the Miller plateau. This slows the transition to the blocking state while the inductor forces its current to commutate (Figure 4.5). These losses grow with frequency and take a large share of the 6 W budget.

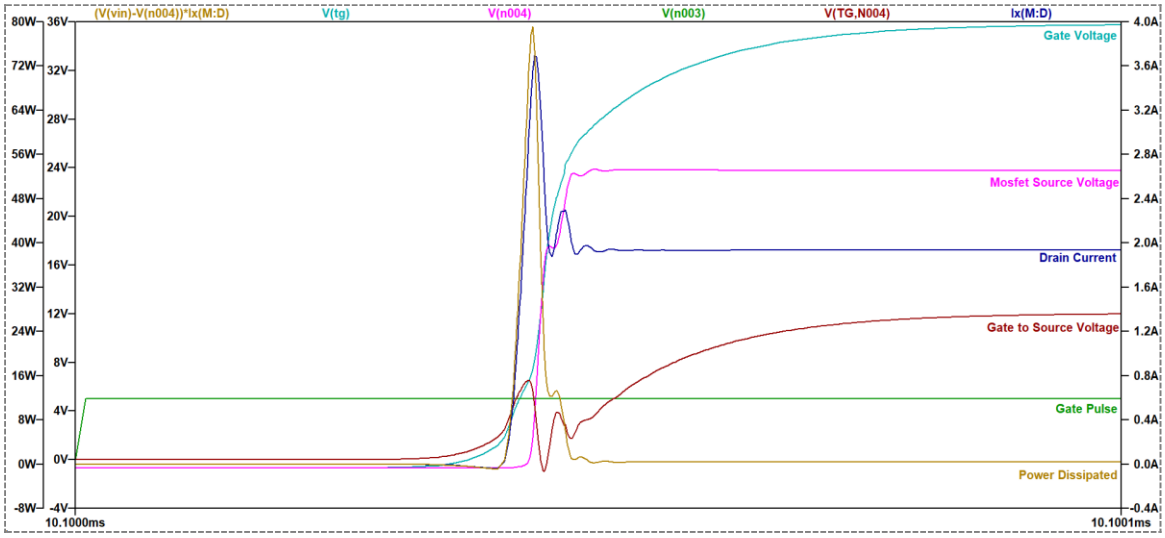


Figure 4.4: Simulated turn-on switching transient

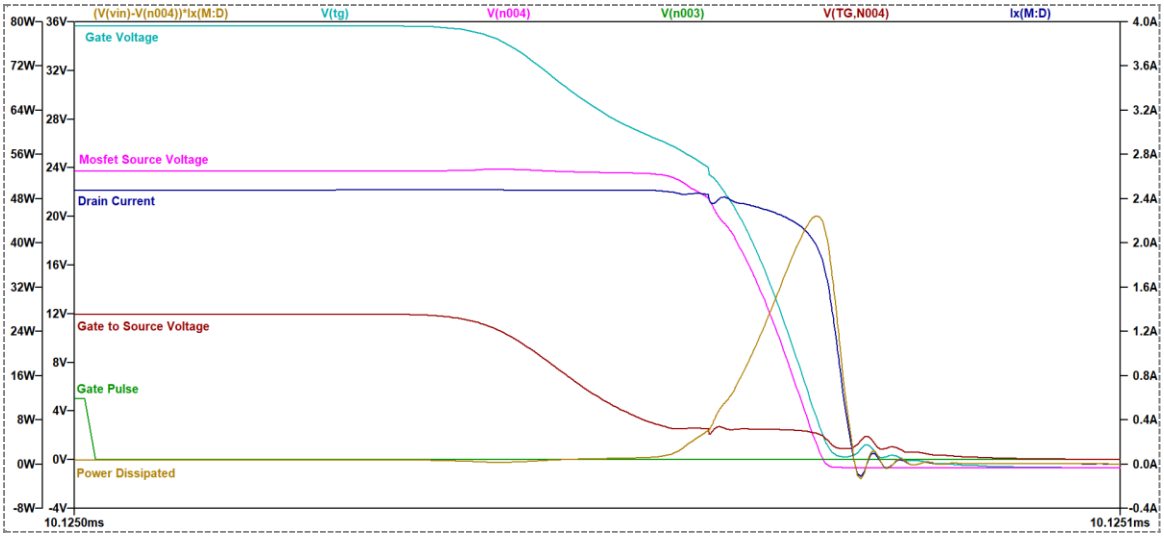


Figure 4.5: Simulated turn-off switching transient

4.4 HARDWARE RESULTS

4.4.1 PCB Design and Prototype

After simulation, the circuit was moved to a PCB. Figure 4.6 shows the schematic, which has a driver circuit and a power circuit. The schematic was checked with the electrical rules check, the board was laid out and checked with the design rules check, and a 3D view was used to confirm component placement before fabrication. The board was then assembled and tested.

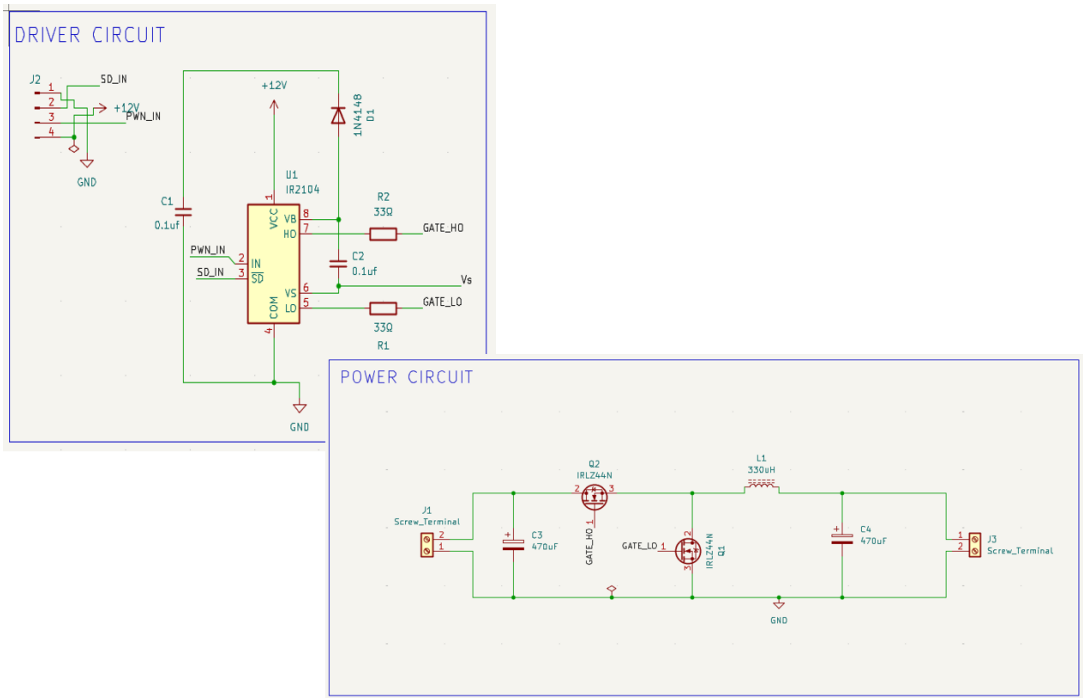


Figure 4.6: PCB schematic: driver circuit and power circuit

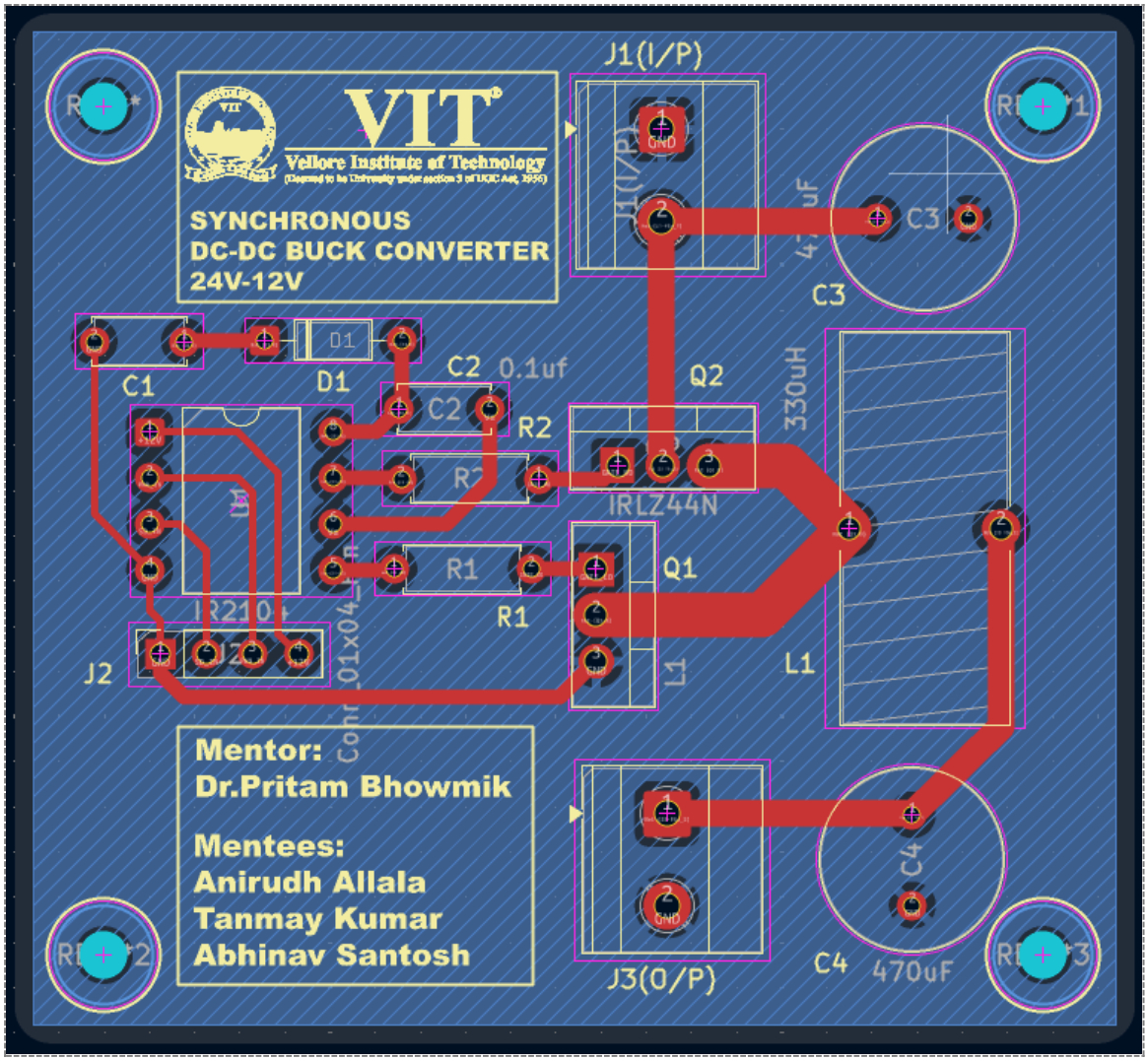


Figure 4.7: PCB layout

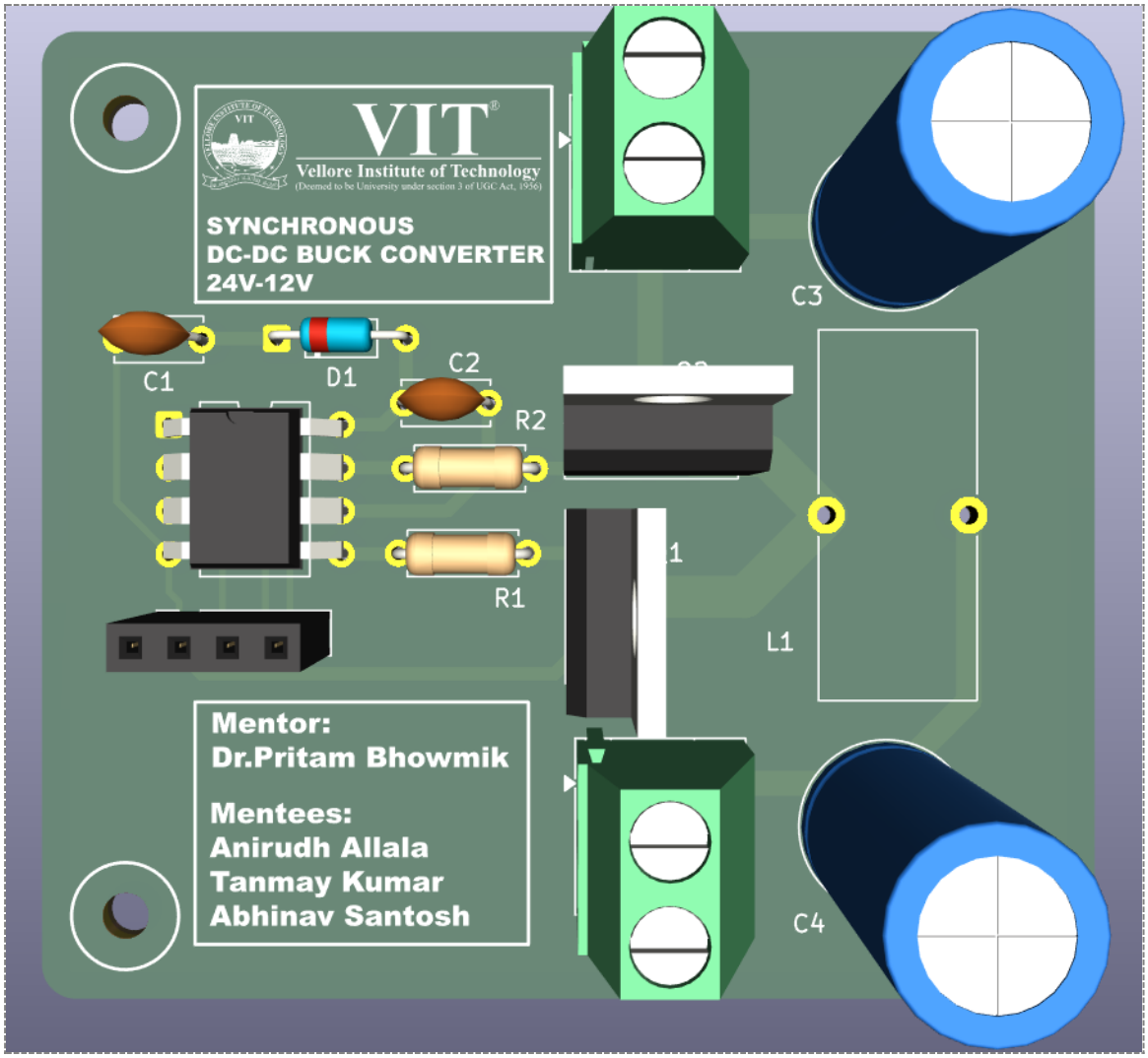


Figure 4.8: PCB 3D view

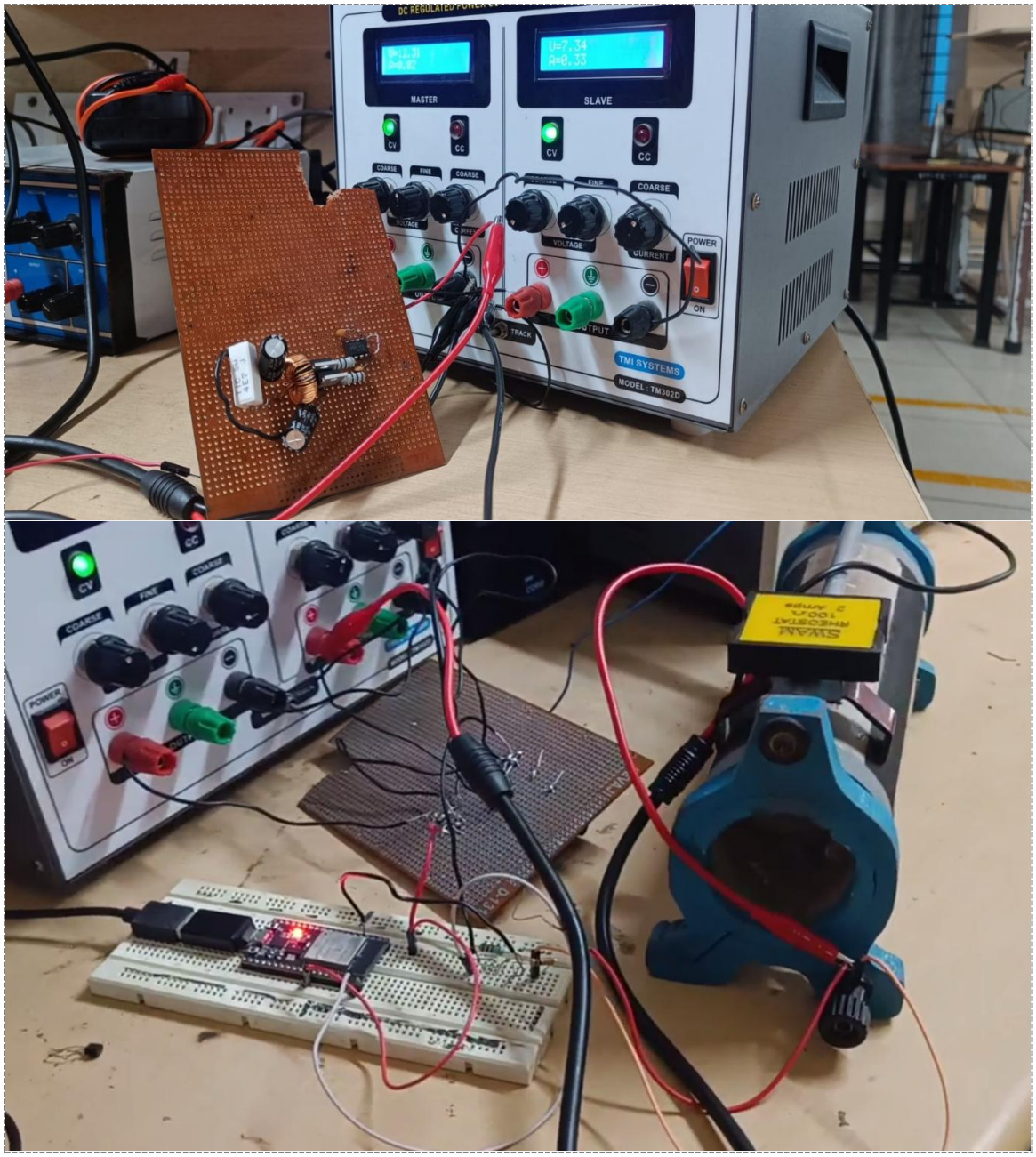


Figure 4.9: Assembled hardware prototype

4.4.2 Switching Waveforms

We recorded the gate-source voltage V_{gs} and the drain-source voltage V_{ds} of the switch driven by the low-side output (LO) at 50 kHz and at 30 kHz. Each plot shows several cycles and then a close-up of a single cycle (Figures 4.10 and 4.11).

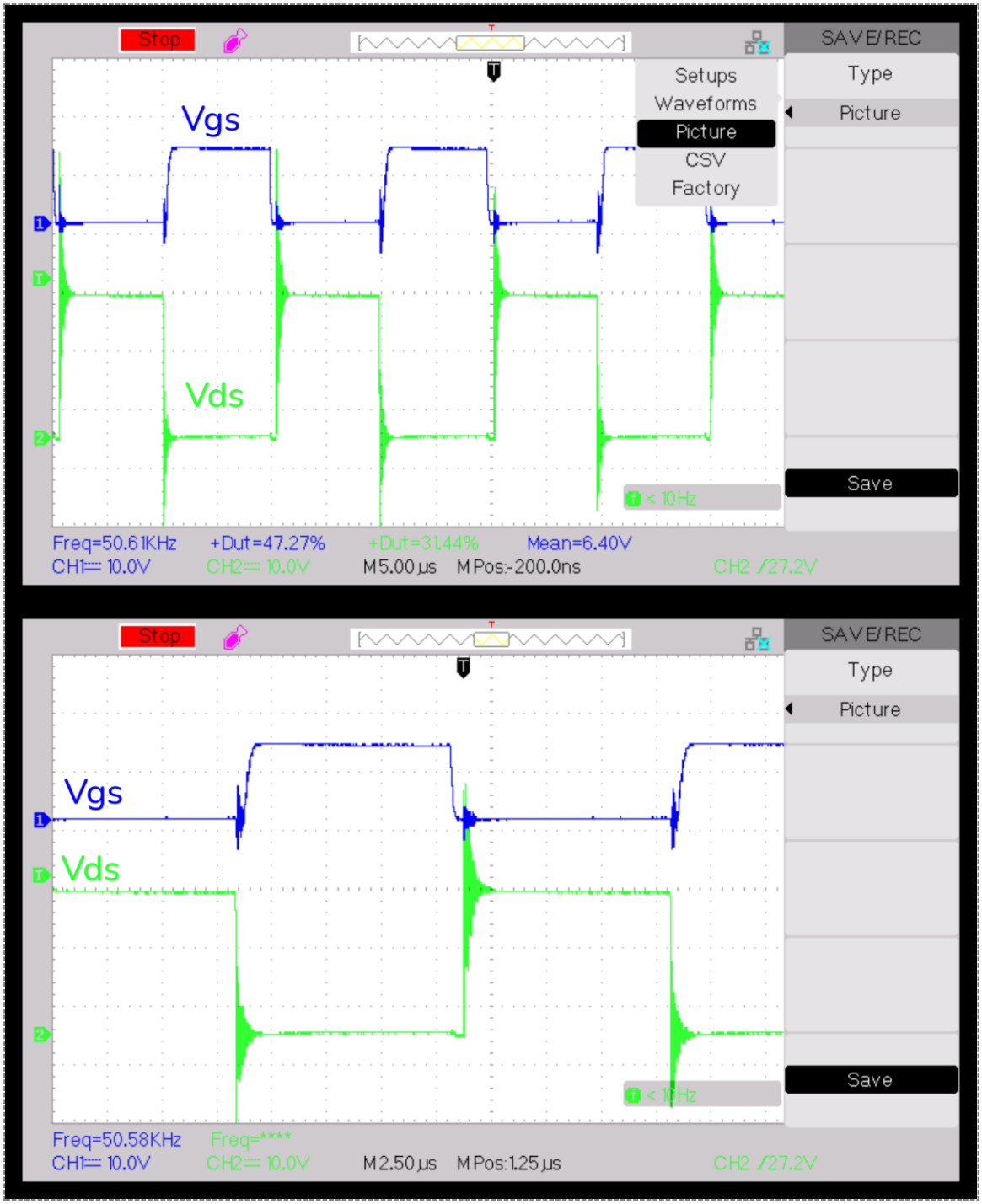


Figure 4.10: Low-side Vgs and Vds at 50 kHz

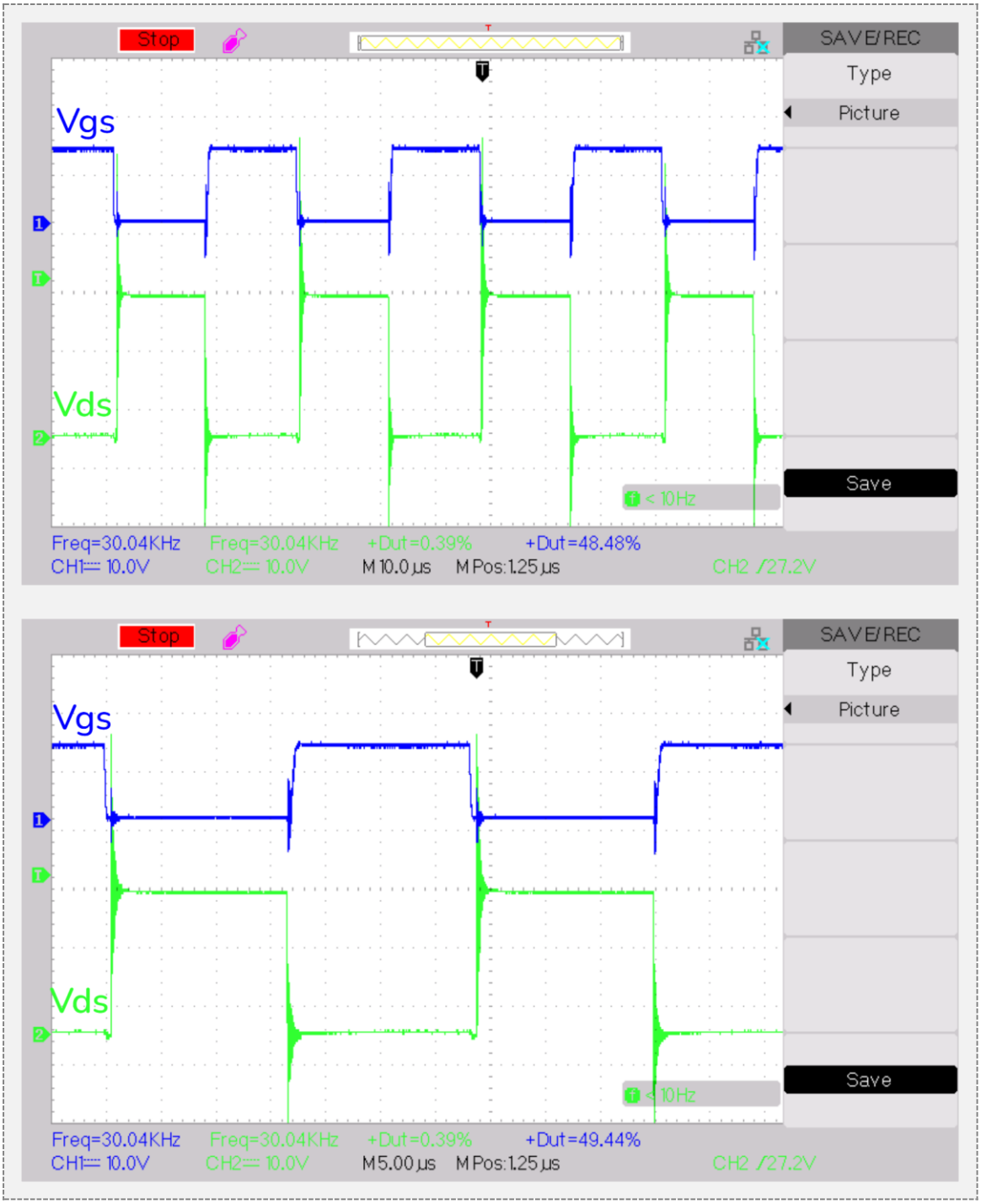
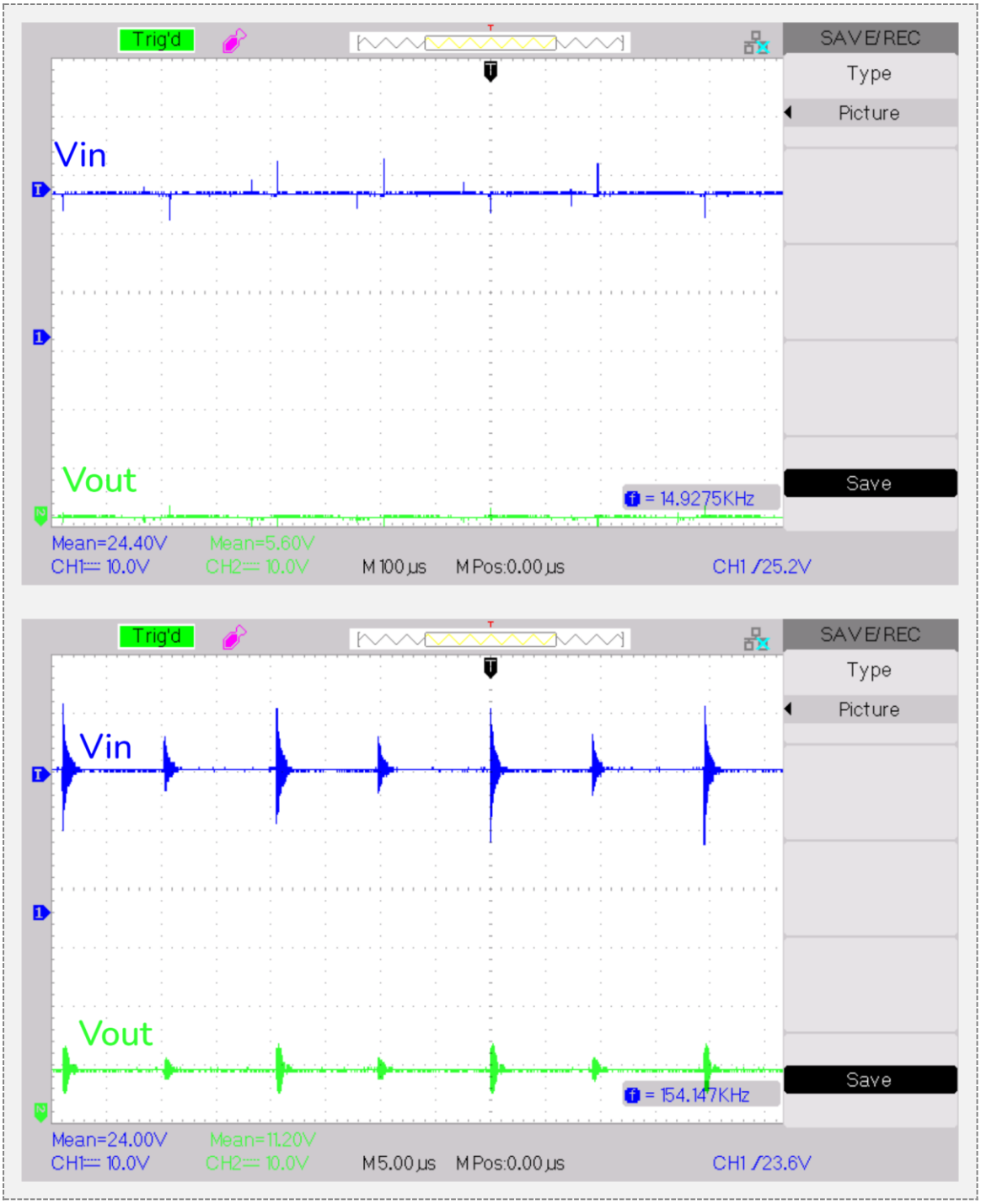


Figure 4.11: Low-side Vgs and Vds at 30 kHz

4.4.3 Input and Output Voltage with Varying Duty Cycle

We changed the duty cycle and recorded the input and output voltages together. The output follows the duty cycle as expected from $V_o = D\ V_{in}$, which shows that the power stage and the PWM chain work correctly (Figure 4.12).


Trig'd
SAVE/REC
Type
Picture
Vin
Vout
Save
= 14.9275KHz
Mean=24.40V
Mean=5.60V
CH1 10.0V
CH2 10.0V
M 100 µs
M Pos:0.00 µs
CH1 /25.2V
Trig'd
SAVE/REC
Type
Picture
Vin
Vout
Save
= 154.147KHz
Mean=24.00V
Mean=11.20V
CH1 10.0V
CH2 10.0V
M 5.00 µs
M Pos:0.00 µs
CH1 /23.6V

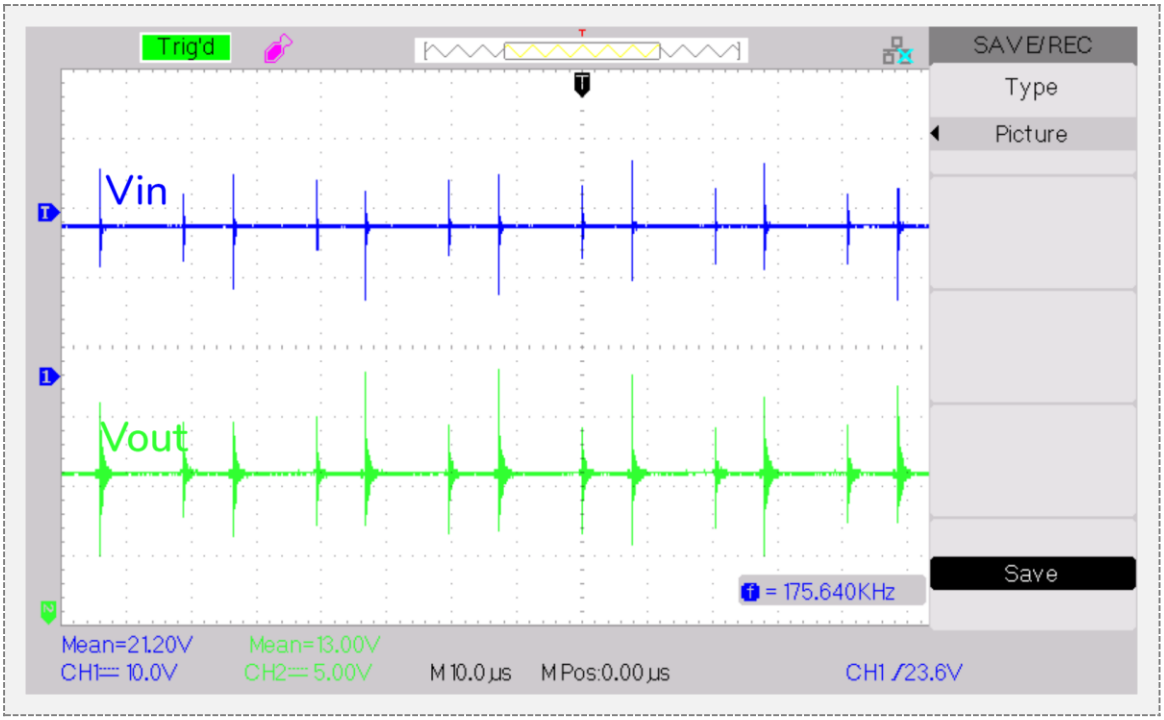


Figure 4.12: Input and output voltage for 25%, 50% and 75% duty cycle respectively

4.4.4 Ringing

A close-up of the input (blue) and output (green) voltage shows ringing at the switching edges. It comes from the parasitic inductance and capacitance of the layout and the switching node. It sets how much voltage margin the MOSFETs need and how much filtering the supply needs (Figure 4.13).

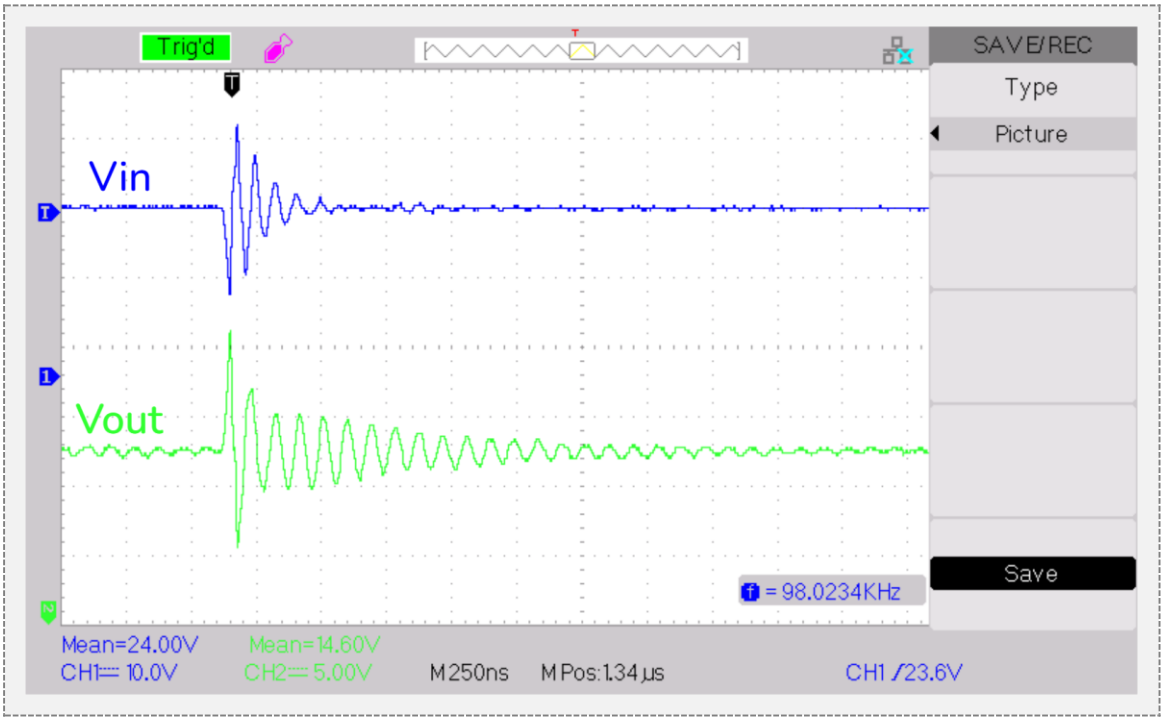


Figure 4.13: Ringing close-up of input and output voltage

4.4.5 Gate Driver Delay and Dead Time

We compared the PWM pulse from the ESP32 (blue) with the output pulse of the gate driver (green). The waveform shows a delay between the two, which comes from the level shifter and the propagation delay of the IR2104. The delay stays the same from cycle to cycle, so at a fixed frequency it only shifts the pulse in time without changing the duty cycle. It does matter for very short pulses. The turn-on and turn-off transients and the dead time between the two MOSFETs were captured as well (Figure 4.14). The measured delay is 440 ns.

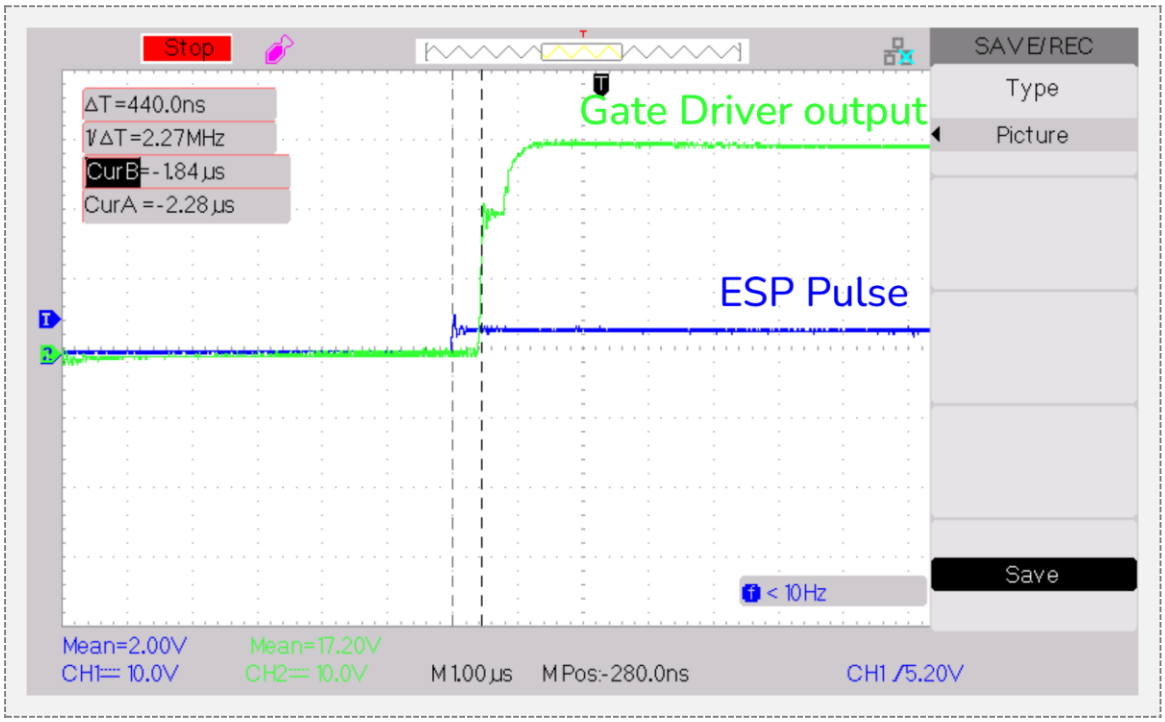


Figure 4.14: ESP32 PWM pulse and gate driver output pulse

4.4.6 Closed-Loop Operation

With the PI controller running, the ESP32 reads the output through the divider, compares it with the 12 V set-point and updates the duty cycle every 100 µs. The firmware prints the target, the measured output voltage, the duty cycle, the PWM count and the raw ADC value at 10 Hz, and these can be watched on the serial plotter. This is the closed-loop buck converter completed for Review 2.

CHAPTER V

CONCLUSION

5.1 COST ANALYSIS

Table 5.1 lists the components of the prototype. The prices are to be filled in from the purchase records.

Table 5.1: Bill of materials and cost

S. No.	Component	Qty	Cost (INR)
1	ESP32 development board	1	₹399.00
2	IR2104 half-bridge gate driver	1	₹98.00
3	IRLZ44N MOSFET	2	₹88.00
4	330 µH inductor	1	₹25.00
5	470 µF electrolytic capacitor	2	₹12.00
6	0.1 µF capacitor	2	N/A
7	1N4148 diode	1	₹2.00
8	33 Ω gate resistor	2	N/A
9	Voltage divider resistors (10 kΩ, 2.2 kΩ)	2	N/A
	Total		₹624.00

5.2 SCOPE OF WORK

The work that remains is listed below.

- Move PWM generation and the voltage and current measurement from the ESP32 to the Arduino Q.
- Add a current sensor and set up the TCP/IP link so that readings go to the host computer and the duty cycle comes back.
- Train and test the ML model that computes the duty cycle from the readings.
- Keep limits and protection on the Arduino Q, with a safe fallback duty cycle if the link is lost or slow.
- Test with two or more converters and batteries at different SoC and see how well they are equalised while a load is running.
- Study how the communication delay affects the stability of the whole system.

5.3 SUMMARY

This project tackles uneven SoC in parallel batteries by controlling the discharge through converters whose duty cycles come from an ML model over TCP/IP. The converter at the base of this system is complete. It is a 24 V to 12 V, 2 A synchronous buck converter built with IRLZ44N switches, a 330 µH inductor, 470 µF capacitors and an IR2104 gate driver. It was designed for 50 kHz, analysed, simulated in MATLAB/Simulink and at circuit level, laid out on a PCB, built, and regulated by a PI controller on an ESP32. Measurements at 50 kHz and 30 kHz show the switching behaviour, the relation between duty cycle and output voltage, the ringing and the driver delay. The TCP/IP link, the ML duty-cycle computation and the SoC equalisation tests will be done in the final phase.

REFERENCES

[1] R. W. Erickson and D. Maksimović, *Fundamentals of Power Electronics*, 3rd ed., Springer, 2020.

[2] N. Mohan, T. M. Undeland and W. P. Robbins, *Power Electronics: Converters, Applications, and Design*, 3rd ed., Wiley, 2003.

[3] D. W. Hart, *Power Electronics*, McGraw-Hill, 2011.

[4] International Rectifier (Infineon Technologies), "IR2104(S) & (PbF) Half-Bridge Driver," datasheet.

[5] Espressif Systems, "ESP32 Series Datasheet" and "Arduino-ESP32 core documentation".

[6] J. Postel, "Transmission Control Protocol," RFC 793, IETF, Sept. 1981.

[7] J. Postel, "Internet Protocol," RFC 791, IETF, Sept. 1981.

[8] W. R. Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*, Addison-Wesley.

[9] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2008.

[10] IPC-2221, *Generic Standard on Printed Board Design*, IPC.

PUBLICATIONS BASED ON THESIS

Journals

Nil.

Conferences

Nil.

APPENDICES

Appendix 1: ESP32 Closed-Loop PI Control Firmware

The following firmware runs on the ESP32 (Arduino core v3.x).

```
// --- ESP32 Closed-Loop Buck Converter PI Control ---
// Updated for ESP32 Arduino Core v3.x

#define V_FB_PIN 34       // ADC input from voltage divider
#define PWM_PIN 18        // PWM output to Level Shifter -> IR2104 IN
#define SD_PIN 19         // Shutdown pin to Level Shifter -> IR2104 SD

// PWM Settings
const int pwmFreq = 50000;      // 50 kHz Switching Frequency
const int pwmResolution = 10;   // 10-bit resolution (0 to 1023 steps)

// Controller Safety Limits
const float MAX_DUTY_PCT = 0.90; // Clamp at 90% to recharge bootstrap cap
const float MIN_DUTY_PCT = 0.00;

// Voltage Divider Calibration
const float R1 = 10000.0;
const float R2 = 2200.0;
const float DIVIDER_RATIO = (R1 + R2) / R2;
const float V_REF = 3.3;        // ESP32 ADC reference voltage
const float ADC_MAX = 4095.0;   // 12-bit ADC

// PI Controller Variables
float V_setpoint = 12.0;        // Target buck output voltage
float Kp = 0.05;                // Proportional gain (Start low!)
float Ki = 0.01;                // Integral gain (Start low!)
float integral = 0.0;

// Telemetry & State Variables (Scoped globally for debug logging)
volatile float V_out = 0.0;
volatile float control_signal = 0.0;
volatile int pwm_steps = 0;
volatile int adc_raw_val = 0;

// Timing for control loop
unsigned long previousMicros = 0;
const unsigned long loopInterval = 100; // 100 us = 10 kHz control loop rate

// Timing for telemetry / debug print
unsigned long previousPrintMillis = 0;
const unsigned long printInterval = 100; // 100 ms = 10 Hz logging rate

void setup() {
  Serial.begin(115200);

  // Setup IR2104 Shutdown pin (Keep LOW/Disabled initially)
  pinMode(SD_PIN, OUTPUT);
  digitalWrite(SD_PIN, LOW);

  // Setup 50 kHz Hardware PWM using the v3.x API
  ledcAttach(PWM_PIN, pwmFreq, pwmResolution);
  ledcWrite(PWM_PIN, 0); // Initialize at 0% duty

  // Configure ADC for full 12-bit range
  analogReadResolution(12);

  // Enable the gate driver
  digitalWrite(SD_PIN, HIGH);
```

```
  Serial.println("System Initialized. Output Enabled.");
}

void loop() {
  unsigned long currentMicros = micros();

  // ------------------------------------------------
  // 1. High-Speed PI Control Loop (10 kHz)
  // ------------------------------------------------
  if (currentMicros - previousMicros >= loopInterval) {
    previousMicros = currentMicros;

    // 1. Read Feedback Voltage
    adc_raw_val = analogRead(V_FB_PIN);
    float v_pin = (adc_raw_val / ADC_MAX) * V_REF;
    V_out = v_pin * DIVIDER_RATIO;

    // 2. Calculate Error
    float error = V_setpoint - V_out;

    // 3. PI Algorithm
    integral += error;
    control_signal = (Kp * error) + (Ki * integral);

    // 4. Anti-Windup Clamping & Limits
    if (control_signal > MAX_DUTY_PCT) {
        control_signal = MAX_DUTY_PCT;
        integral -= error;
    }
    else if (control_signal < MIN_DUTY_PCT) {
        control_signal = MIN_DUTY_PCT;
        if (error < 0) integral -= error;
    }

    // 5. Apply PWM Update
    pwm_steps = (int)(control_signal * 1023.0f);
    ledcWrite(PWM_PIN, pwm_steps);
    // If your driver requires inverted PWM polarity, use:
    // ledcWrite(PWM_PIN, 1023 - pwm_steps);
  }

  // ------------------------------------------------
  // 2. Decoupled Debug / Telemetry Stream (10 Hz)
  // ------------------------------------------------
  unsigned long currentMillis = millis();
  if (currentMillis - previousPrintMillis >= printInterval) {
    previousPrintMillis = currentMillis;

    // Serial Plotter-compatible format (Label:Value)
    Serial.print("Target:");
    Serial.print(V_setpoint, 2);
    Serial.print(" V_out:");
    Serial.print(V_out, 2);
    Serial.print(" DutyPct:");
    Serial.print(control_signal * 100.0, 1);
    Serial.print(" PWM:");
    Serial.print(pwm_steps);
    Serial.print(" ADC:");
    Serial.println(adc_raw_val);
  }
}
```

Appendix 2: Simulation Model Description

Table A.1 relates each block of the MATLAB/Simulink model in Figure 4.1 to its function.

Table A.1: Simulink model blocks

Block	Function
DC voltage source	24 V input supply
High-side MOSFET	Main switch, gate driven by the PWM signal
NOT gate	Inverts the PWM to drive the low-side MOSFET complementarily
Low-side MOSFET with diode	Synchronous rectifier and freewheeling path
Inductor and current measurement	Output filter inductor and its current
Capacitor and load resistor	Output filter capacitor and load
Voltage measurement and scope	Output voltage sensing, feedback and display
Constant (12)	Voltage reference
Sum and PI controller with saturation	Error computation and PI regulation of the duty cycle
PWM generator	Converts the duty cycle to the gate pulse
powergui (continuous)	Continuous solver for the power electronic circuit

CURRICULUM VITAE

Abhinav Santosh Kumar (23BEE1023)

Field	Details
Name	Abhinav Santosh Kumar
Register Number	23BEE1023
Date of Birth	
Email	
Address for Communication	
Education	B.Tech, Electrical and Electronics Engineering, VIT Chennai
Interests	

Tanmay Kumar (23BEE1105)

Field	Details
Name	Tanmay Kumar
Register Number	23BEE1105
Date of Birth	
Email	
Address for Communication	
Education	B.Tech, Electrical and Electronics Engineering, VIT Chennai
Interests	

Anirudh Allala (23BEE1111)

Field	Details
Name	Anirudh Allala
Register Number	23BEE1111
Date of Birth	
Email	
Address for Communication	
Education	B.Tech, Electrical and Electronics Engineering, VIT Chennai
Interests	